Cost-Efficient Foreground Segmentation Using a Lightweight Depthwise-Separable U-Net under a Single-Core CPU Budget

Rahul Kumar Bhikundia
Roll Number: M26CSA006
CSL7360: Computer Vision — Assignment 2
Indian Institute of Technology Jodhpur

Abstract

This report presents a technical overview and experimental evaluation of a cost-efficient binary foreground segmentation pipeline designed to segment the primary object of interest within a 256×256 image under a strict single-core CPU latency budget ($\leq$ 50 ms per image) and compact storage footprint. Following Track B, a lightweight U-Net-inspired fully convolutional neural network (`SmallUNet`) was designed and trained completely from scratch on the DUTS-TR dataset without relying on pretrained weights. The architecture combines depthwise-separable convolutions, batch normalization, bilinear upsampling, skip connections, and internal 192×192 spatial resolution scaling, resulting in a compact model with $29,617$ trainable parameters occupying $185,126$ bytes (≈ 0.1765 MiB) on disk. Evaluated across the 5,019-image DUTS-TE test set under single-threaded CPU execution, the pipeline achieves a mean Intersection over Union (mIoU) of 0.63016, a Boundary F1-score (BF1) of 0.28823, and a quality score of $Q = 0.52758$ at an average CPU inference time of 45.8819 ms, yielding a final Cost-Efficient Segmentation (CES) score of 0.42843.

1 Objective

The objective of this work is to design and train from scratch a computationally efficient binary foreground segmentation pipeline that assigns a binary class label (1 for main object, 0 for background) to every pixel of an input photograph while operating within a strict single-threaded CPU compute budget. Specifically, the target pipeline must satisfy the following requirements:

- Achieve an average single-core CPU inference latency of at most 50 ms per photograph across the 5,019-image DUTS-TE test benchmark.
- Train a small Fully Convolutional Network (FCN) completely from scratch using only primitive PyTorch layers without pretrained weights or external segmentation libraries.
- Maximize the Cost-Efficient Segmentation (CES) metric by balancing region overlap (mIoU), contour accuracy (Boundary F1), single-threaded CPU runtime (T_{CPU}), and parameter file size (S_{disk}).
- Generalize across challenging real-world campus and hostel scenes exhibiting indoor and outdoor lighting, scale variations, cluttered backgrounds, and partial occlusions.

2 Overall Pipeline Architecture

The segmentation pipeline is divided into offline image resizing, colour conversion and tensor normalization, internal 192×192 bilinear downscaling, three-stage depthwise-separable U-Net

encoding and decoding with skip connections, output upscaling to 256×256, and direct zero-logit thresholding. Figure 1 illustrates the complete execution dataflow, highlighting the operations included within the timed single-core CPU benchmark.

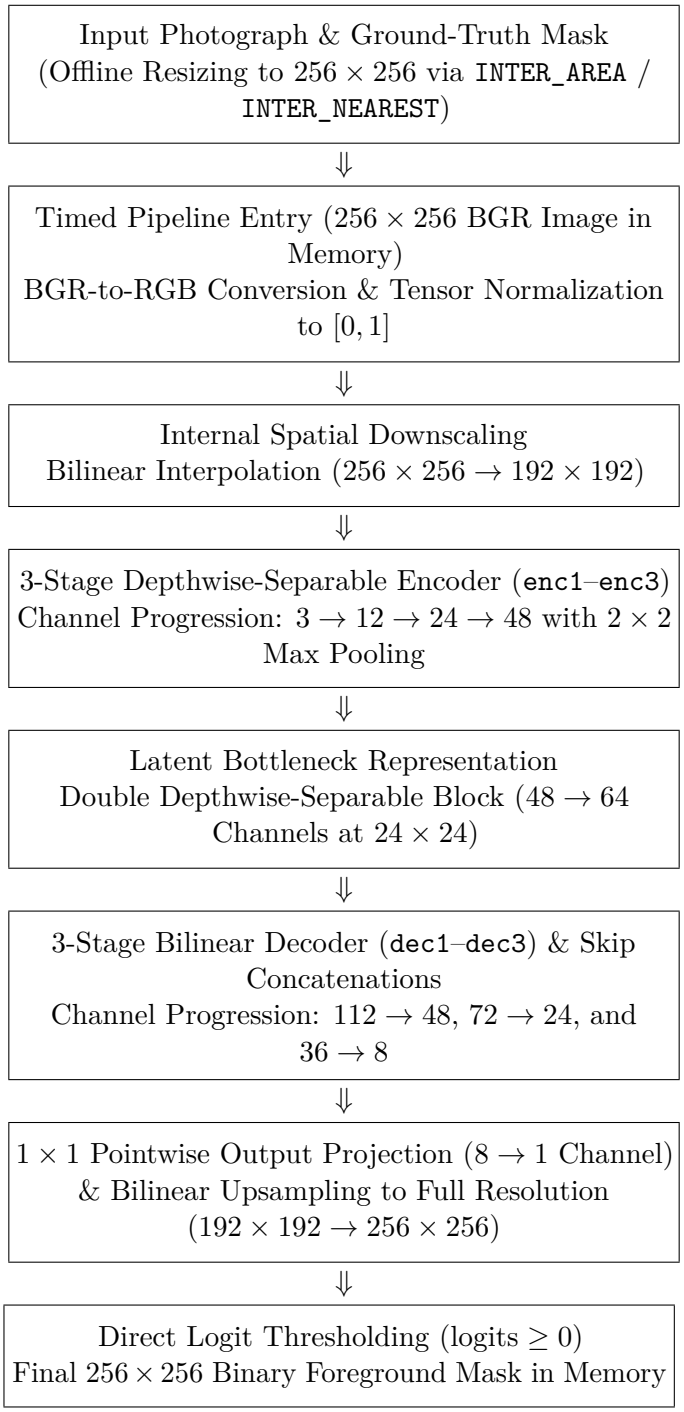


Figure 1: System architecture and timed execution dataflow of the `SmallUNet` segmentation pipeline.

3 Dataset and Preprocessing

3.1 DUTS Dataset Partitioning

The DUTS salient object detection dataset was utilized for model training, validation, and official benchmarking. To ensure strict separation between model selection and final evaluation, the 10,553-image `DUTS-TR` dataset was partitioned into a 90% training subset and a 10% validation subset using a fixed random seed (`torch.Generator().manual_seed(42)`). The official `DUTS-TE` test dataset was reserved exclusively for final evaluation after freezing the selected model checkpoint:

- **DUTS-TR Training Subset:** $9,497$ image–mask pairs, Tensor shapes: $(3, 256, 256)$ and $(1, 256, 256)$
- **DUTS-TR Validation Subset:** $1,056$ image–mask pairs, used for hyperparameter tuning and checkpoint selection
- **DUTS-TE Official Test Set:** $5,019$ image–mask pairs, reserved strictly for final testing and CPU benchmarking

3.2 Image and Mask Resizing Protocol

All images and ground-truth segmentation masks are standardized to a spatial resolution of 256×256 pixels according to Section 1b of the specification:

1. Input photographs are read using OpenCV and resized to 256×256 using pixel-area interpolation (`cv2.INTER_AREA`).
2. Ground-truth grayscale masks are resized to 256×256 using nearest-neighbor interpolation (`cv2.INTER_NEAREST`) to preserve discrete boundary labels without introducing intermediate gray values.
3. Mask pixels satisfying intensity ≥ 128 are assigned to the foreground object class (1.0), and pixels < 128 are assigned to the background class (0.0).
4. Input images are converted from BGR to RGB format, permuted to channel-first tensor format $(3, 256, 256)$, and normalized to the range $[0, 1]$ by dividing by 255.0.

3.3 Personal Photograph Dataset (`captured/`)

To evaluate out-of-distribution generalization on real-world scenes, six photographs were captured in campus and hostel environments covering indoor and outdoor scenes, daylight and tube lighting, near and far objects, cluttered backgrounds, and partial occlusion. Via the preparation script (`prepare_captured.py`), the dataset was structured into three subdirectories (`original/`, `images_256/`, and `masks_256/` inside `captured/`) with matching base filenames (`photo1` through `photo6`). The six scenes depict a potted plant (`photo1`), a matte black water bottle on a cluttered desk (`photo2`), a sneaker on an indoor floor (`photo3`), a partially occluded biscuit packet behind an electronic controller/monitor unit (`photo4`), a distant outdoor water dispenser (`photo5`), and a bicycle parked against a brick wall (`photo6`).

4 Proposed Network Architecture (`SmallUNet`)

To satisfy the 50 ms single-core CPU inference budget, a lightweight U-Net-inspired encoder–decoder architecture (`SmallUNet`) was implemented from scratch in PyTorch. Rather than relying on standard 3×3 convolutions, which require $9\,C_{\text{in}}C_{\text{out}}HW$ multiply-accumulate operations

(MACs), each convolutional stage factorizes spatial filtering and channel mixing into a **depthwise-separable convolution**:

1. **Depthwise Convolution:** Applies a 3×3 spatial filter independently to each input channel (padding $= 1$, groups $= C_{\text{in}}$), followed by BatchNorm2d(C_{in}) and in-place ReLU activation.

2. **Pointwise Convolution:** Applies a 1×1 convolution across channels ($C_{\text{in}} \to C_{\text{out}}$), followed by BatchNorm2d(C_{out}) and in-place ReLU activation.

Each standard encoder and decoder block (`ConvBlock`) stacks two consecutive depthwise-separable units, whereas the highest-resolution final decoder stage (`FinalDecoderBlock`) uses a single depthwise-separable unit ($36 \to 8$ channels) followed by a 1×1 output convolution ($8 \to 1$ channel) to minimize full-resolution compute overhead. Furthermore, `SmallUNet.forward()` bilinearly downsamples the 256×256 input tensor to 192×192 prior to feature extraction and bilinearly upsamples the output logit map back to 256×256, reducing spatial convolution operations by $1 - (192/256)^2 = 43.75\%$ within the timed execution block. Table 1 summarizes the layer-by-layer architecture and parameter count.

Table 1: Layer-by-layer specification and trainable parameter breakdown of `SmallUNet`.

Stage	**Block / Operation**	**Channels ($C_{\text{in}} \to C_{\text{out}}$)**	**Spatial Resolution**	**Parameters**
Input Resize	Bilinear Downsample (`F.interpolate`)	$3 \to 3$	$256 \times 256 \to 192 \times 192$	0
`enc1 + pool1`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$3 \to 12$	$192 \times 192 \to 96 \times 96$	432
`enc2 + pool2`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$12 \to 24$	$96 \times 96 \to 48 \times 48$	1,440
`enc3 + pool3`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$24 \to 48$	$48 \times 48 \to 24 \times 24$	4,608
`bottleneck`	`ConvBlock` (2× DW-Sep)	$48 \to 64$	24×24	8,896
`up1 + dec1`	2× Bilinear + Concat(`enc3`) + `ConvBlock`	$64 + 48 = 112 \to 48$	$24 \times 24 \to 48 \times 48$	9,888
`up2 + dec2`	2× Bilinear + Concat(`enc2`) + `ConvBlock`	$48 + 24 = 72 \to 24$	$48 \times 48 \to 96 \times 96$	3,600
`up3 + dec3`	2× Bilinear + Concat(`enc1`) + `FinalDecoderBlock`	$24 + 12 = 36 \to 8$	$96 \times 96 \to 192 \times 192$	744
`out` + Upsample	1×1 Conv + Bilinear Upsample	$8 \to 1$	$192 \times 192 \to 256 \times 256$	9
Total	**Complete Trainable Network**	—	$\mathbf{256 \times 256 \to 256 \times 256}$	**29,617**

5 Training Configuration and Checkpoint Selection

The network was trained from scratch on the 9,497-image DUTS-TR training split using an NVIDIA GTX 1650 GPU. Optimization was performed over 10 epochs with a batch size of 16 using the Adam optimizer (lr $= 0.001$) and pixel-wise Binary Cross-Entropy with Logits Loss (`nn.BCEWithLogitsLoss()`):

$$\mathcal{L}_{\text{BCE}}(\hat{z}, y) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log \sigma(\hat{z}_i) + (1 - y_i) \log \big(1 - \sigma(\hat{z}_i)\big) \Big] \tag{1}$$

where $\hat{z}_i$ denotes the predicted logit at pixel i, $y_i \in \{0, 1\}$ denotes the ground-truth label, and $\sigma(\hat{z}_i) = (1 + e^{-\hat{z}_i})^{-1}$. During inference, foreground classification is performed via direct logit thresholding ($\hat{z}_i \geq 0$), which is equivalent to $\sigma(\hat{z}_i) \geq 0.5$ and avoids computing the sigmoid function across all $65,536$ pixels.

Validation loss was evaluated after every epoch on the 1,056-image validation set, decreasing overall from 0.3612 in Epoch 1 to a minimum of 0.2587 in Epoch 10. The checkpoint achieving the lowest validation loss was saved to `checkpoints/best_model_192.pth` (185,126 bytes) and frozen for final evaluation. Table 2 summarizes the training configuration.

Table 2: Summary of training hyperparameters and checkpoint selection.

Configuration Parameter	**Value / Specification**
Framework	PyTorch (Track B FCN from scratch)
Training / Validation Split	$9,497$ Training / $1,056$ Validation (Seed = 42)
Input / Internal Resolution	256×256 Input / 192×192 Internal
Batch Size / Epochs	16 / 10 Epochs
Optimizer / Learning Rate	Adam / 1.0×10^{-3}
Loss Function	Binary Cross-Entropy with Logits (`BCEWithLogitsLoss`)
Validation Loss Trajectory	0.3612 (Epoch 1) $\rightarrow$ 0.2587 (Epoch 10, Best)
Training Hardware	NVIDIA GeForce GTX 1650 GPU
Saved Parameter File	`checkpoints/best_model_192.pth` ($185,126$ bytes)

6 Evaluation Metrics

6.1 Mean Intersection over Union (mIoU)

To evaluate region-level overlap, true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN) are accumulated into a single global 2×2 confusion matrix across all images in an evaluation set. The class-wise IoU values for background and foreground and their mean (mIoU) are defined as:

$$\mathrm{IoU}_{\mathrm{bg}} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP} + \mathrm{FN}}, \qquad \mathrm{IoU}_{\mathrm{fg}} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}, \qquad \mathrm{mIoU} = \frac{\mathrm{IoU}_{\mathrm{bg}} + \mathrm{IoU}_{\mathrm{fg}}}{2} \tag{2}$$

6.2 Boundary F1-Score (BF1)

To evaluate contour alignment, a foreground pixel is defined as a boundary pixel if at least one of its 4-connected neighbors (up, down, left, right) is a background pixel, with pixels outside the 256×256 image frame treated as background. A predicted boundary pixel is considered correct if a ground-truth boundary pixel lies within its 5×5 window, and a ground-truth boundary pixel is considered found if a predicted boundary pixel lies within its 5×5 window. In `evaluate.py` and `evaluation_test.py`, both 4-neighbor boundary extraction and 5×5 window dilation are implemented via vectorized NumPy boolean array shifts. Boundary precision P, recall R, and per-image BF1 are computed as:

$$P = \frac{\text{Matched Predicted Boundary Pixels}}{\text{Total Predicted Boundary Pixels}}, \qquad R = \frac{\text{Matched True Boundary Pixels}}{\text{Total True Boundary Pixels}} \tag{3}$$

$$\mathrm{BF1} = \frac{2PR}{P + R} \tag{4}$$

and averaged across all images in the evaluation set.

6.3 Quality Score (Q) and Cost-Efficient Segmentation (CES) Score

The combined segmentation quality score Q and overall Cost-Efficient Segmentation (CES) score are given by:

$$Q = 0.70\,\mathrm{mIoU} + 0.30\,\mathrm{BF1} \tag{5}$$

$$\mathrm{CES} = \begin{cases} Q\left[1 - 0.20\,\dfrac{T_{\mathrm{CPU}}}{50\ \mathrm{ms}} - 0.10\,\min\left(1, \dfrac{S_{\mathrm{disk}}}{4\ \mathrm{MiB}}\right)\right] & \text{if } T_{\mathrm{CPU}} \leq 50\ \mathrm{ms} \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

where 4 MiB = $4,194,304$ bytes.

7 Single-Threaded CPU Benchmarking Protocol

In `benchmark.py` and `predict.py`, strict single-threaded CPU execution is enforced before importing numerical libraries by setting `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `VECLIB_MAXIMUM_THREADS` to `"1"`, and configuring PyTorch and OpenCV thread pools to a single thread via `torch.set_num_threads(1)`, `torch.set_num_interop_threads(1)`, and `cv2.setNumThreads(1)`. After 10 warm-up runs, each pre-resized 256×256 image in memory is timed individually using `time.process_time()`, covering BGR-to-RGB colour conversion, tensor preparation, `SmallUNet` forward inference (including 192×192 scaling), and binary mask conversion in memory.

8 Experimental Results

The quantitative segmentation results across the DUTS-TR validation split, the official 5,019-image DUTS-TE test set, and the six captured campus/hostel photographs are summarized in Table 3. Detailed single-core CPU timing and storage statistics on DUTS-TE are listed in Table 4.

Table 3: Overall segmentation quality and cost-efficiency metrics across all evaluation datasets.

Evaluation Split	Images	BG IoU	FG IoU	mIoU	BF1	Quality (Q)	$\mathbf{T_{CPU}}$ (ms)	$\mathbf{S_{disk}}$ (B)	CES
DUTS-TR Validation	1,056	0.85963	0.65198	0.75581	0.38706	0.64518	—	185,126	—
DUTS-TE Official Test	5,019	0.85440	0.40592	0.63016	0.28823	0.52758	45.8819	185,126	**0.42843**
Captured Photos	6	0.91745	0.55856	0.73800	0.39868	0.63621	45.8819	185,126	—

Table 4: Official single-threaded CPU benchmark summary across all 5,019 DUTS-TE test images.

Benchmark Measurement	Measured Result
Total Test Images Processed (`DUTS-TE`)	5,019
Average CPU Inference Time (T_{CPU})	45.8819 ms
Median CPU Inference Time	46.8750 ms
Maximum CPU Inference Time	140.6250 ms
Saved Checkpoint Size (S_{disk})	185,126 bytes (≈ 0.1765 MiB)
Time Penalty Factor ($0.20 \times T_{\text{CPU}}/50$ ms)	0.18353
Size Penalty Factor ($0.10 \times S_{\text{disk}}/4$ MiB)	0.00441
Final Cost-Efficient Segmentation (CES) Score	**0.42843**

Substituting the official DUTS-TE test set values ($Q = 0.52758$, $T_{\text{CPU}} = 45.8819$ ms ≤ 50 ms, and $S_{\text{disk}} = 185,126$ bytes) into the CES equation yields:

$$\text{CES} = 0.52758 \times \left(1 - 0.20 \times \frac{45.8819}{50.0} - 0.10 \times \frac{185,126}{4,194,304}\right) = 0.52758 \times 0.812059 = 0.42843 \quad (7)$$

9 Personal Photograph Evaluation and Visualizations

Table 5 lists the per-image segmentation metrics across all six captured photographs. For each photograph, `visualize.py` generates a four-panel comparison showing the original image, the ground-truth foreground overlay (semi-transparent red), the predicted foreground overlay (semi-transparent green), and a pixel agreement map (where yellow indicates true-positive overlap, red indicates missed foreground pixels, and green indicates false-positive background pixels). The visual comparisons are presented in Figures 2 and 3.

Table 5: Per-image performance metrics across the six captured campus and hostel photographs.

Photo ID	Object	Scene Conditions	BG IoU	FG IoU	mIoU	BF1	Quality (Q)
`photo1`	Plant	Outdoor, daylight, near	0.9527	0.7599	0.8563	0.5181	0.7548
`photo2`	Bottle	Indoor, tube light, clutter	0.9426	0.5733	0.7580	0.4119	0.6541
`photo3`	Shoe	Indoor, tube light, near	0.9872	0.9253	0.9563	0.8378	0.9207
`photo4`	Occluded Packet	Indoor, partial occlusion	0.7897	0.3954	0.5925	0.1791	0.4685
`photo5`	Water Dispenser	Outdoor, far/small object	0.9564	0.1645	0.5604	0.0661	0.4121
`photo6`	Bicycle	Outdoor, thin structures	0.8724	0.4641	0.6683	0.3792	0.5815
Aggregate	**All 6 Photos**	**Global mIoU & Mean BF1**	**0.91745**	**0.55856**	**0.73800**	**0.39868**	**0.63621**

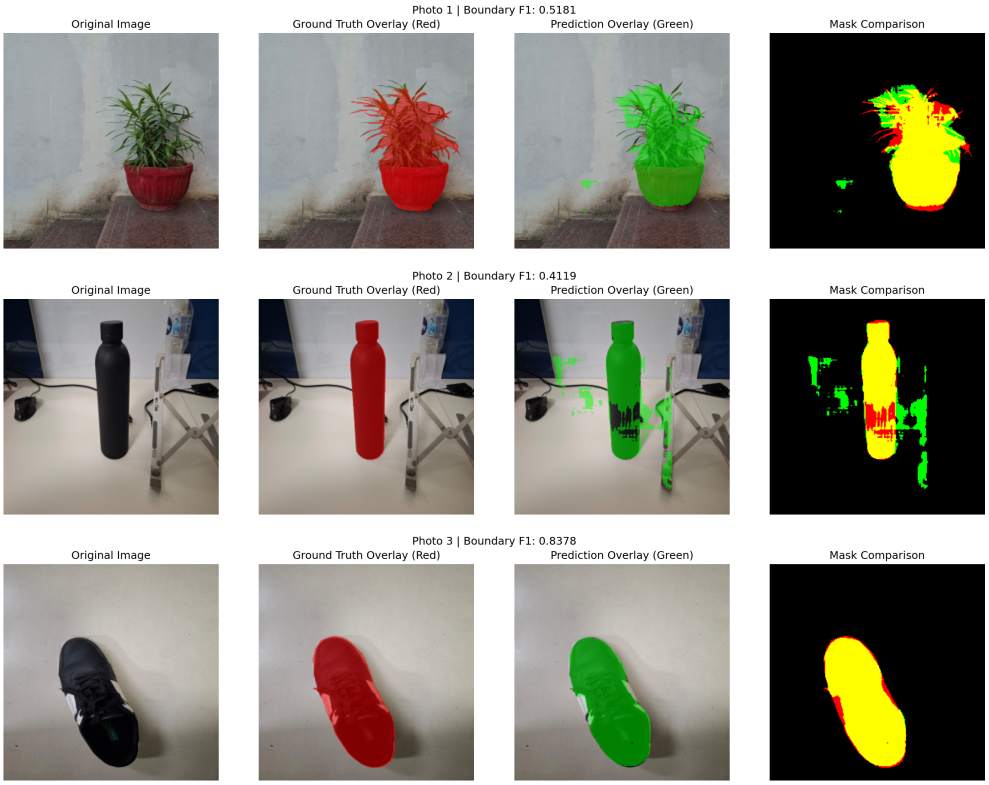


Figure 2: Four-panel qualitative segmentation comparisons for `photo1` (plant), `photo2` (water bottle), and `photo3` (shoe).

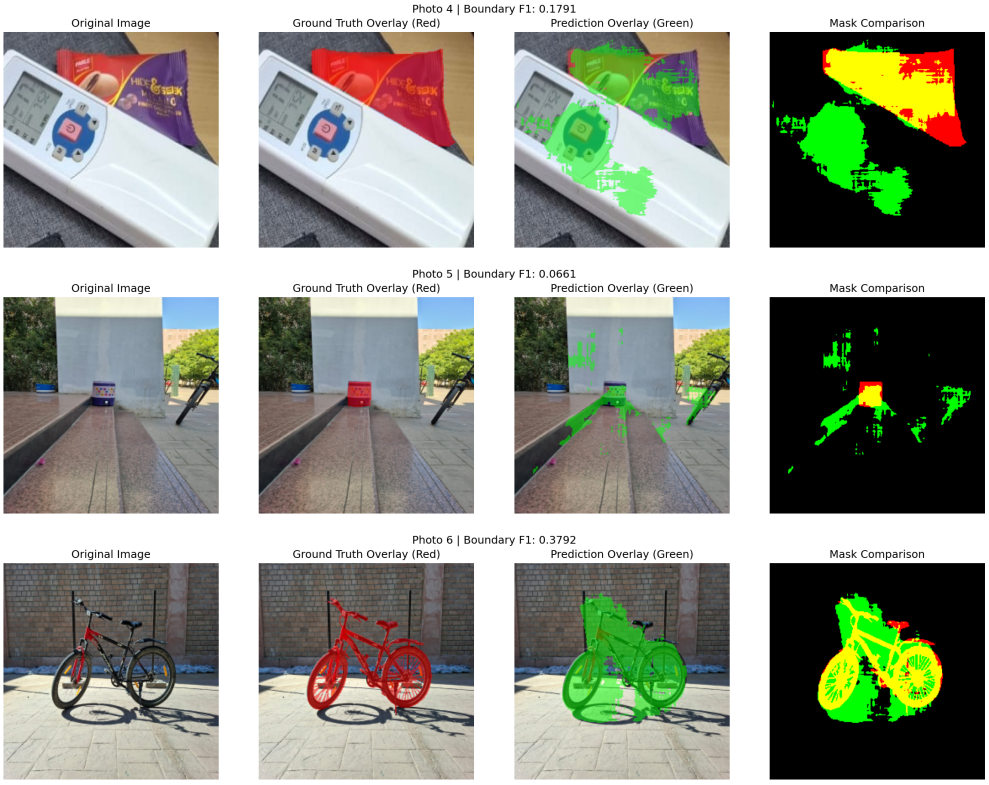


Figure 3: Four-panel qualitative segmentation comparisons for `photo4` (occluded packet/controller), `photo5` (distant water dispenser), and `photo6` (bicycle).

10 Discussion and Failure Case Analysis

Successful Segmentation (`photo3` and `photo1`): The shoe photograph (`photo3`) achieved the highest performance (mIoU = 0.9563, BF1 = 0.8378) due to strong luminance contrast between the dark shoe and the uniform indoor floor. Similarly, the potted plant (`photo1`, mIoU = 0.8563, BF1 = 0.5181) was accurately segmented across the pot and central foliage, losing precision primarily along fine overlapping leaf tips.

Failure Case 1 — Partial Occlusion and Multi-Component Appearance (`photo4`, BF1 = 0.1791): In `photo4`, the purple biscuit packet (17.31% true foreground area) is partially occluded in the foreground by a bright white electronic remote/controller unit containing a high-contrast LCD and circular button pad. Because the network is trained for general foreground saliency without semantic object category priors, it activates across both the upper packet and the high-contrast button pad of the occluding controller (25.36% predicted foreground area), resulting in a fragmented boundary and elevated false positives (BG IoU = 0.7897).

Failure Case 2 — Small Object Scale and Background Clutter (`photo5`, BF1 = 0.0661): In `photo5`, the target water dispenser occupies only 1.23% of the image area at a far distance. After internal downsampling to 192×192 and three 2×2 max-pooling stages, the bottleneck feature map has a spatial resolution of 24×24, where the dispenser spans fewer than 3×3 cells. Consequently, fine object contours are attenuated during encoding, and high-contrast background edges (the step riser and adjacent bicycle wheel) produce false-positive activations.

Failure Case 3 — Thin Geometric Structures and Specular Clutter (`photo6`, BF1 = 0.3792; `photo2`, BF1 = 0.4119): In `photo6`, spatial downsampling and bilinear upsampling blur across the bicycle's narrow frame tubes and wheel spokes, causing the decoder to fill the interior wheel regions as solid foreground blobs (FG IoU = 0.4641). In `photo2`, indoor tube-light reflection bisects the black bottle surface while nearby desk objects (mouse and laptop stand) generate false positives.

11 Conclusion

An end-to-end lightweight binary foreground segmentation framework (`SmallUNet`) was designed, trained from scratch on DUTS-TR, and evaluated under a strict single-core CPU budget. By combining depthwise-separable convolutions with internal 192×192 spatial scaling, the $29,617$-parameter network ($185,126$ bytes) processes 256×256 images in 45.8819 ms on a single CPU thread and achieves mIoU = 0.63016, BF1 = 0.28823, $Q = 0.52758$, and CES = 0.42843 on the 5,019-image DUTS-TE benchmark.